



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G4

System Integration and Architecture

System Design Document (SDD)

Project Title: RevnU: A Sales Management System for
Restaurants

Prepared By: Bigno, Richemmae Valle

Version: 2.0

Date: 04/28/2026

Status: Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	02/18/2026	Richemmae Bigno	Initial draft	Draft
1.1	02/23/2025	Richemmae Bigno	Added changes to scope, added erd and component diagram	Draft
1.2	03/03/2025	Richemmae Bigno	Finalized single-tenant logic; updated ERD with Payouts, Restaurant Config, and expanded User profiles; updated MoSCoW and User Journeys.	Draft
2.0	04/16/2025	Richemmae Bigno	Updated Project Description	Draft

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY.....	4
1.0 INTRODUCTION.....	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION.....	5
3.0 NON-FUNCTIONAL REQUIREMENTS.....	8
4.0 SYSTEM ARCHITECTURE.....	9
5.0 API CONTRACT & COMMUNICATION.....	10
Authentication Endpoints.....	11
User Registration.....	11
User Login.....	11
6.0 DATABASE DESIGN.....	13
7.0 UI/UX DESIGN.....	14
8.0 PLAN.....	17

EXECUTIVE SUMMARY

1.1 Project Overview

RevnU is a restaurant management system designed to digitize and streamline financial and operational tracking for individual restaurant owners (Tenants). The system enables tenants to record sales, expenses, and personal business metrics while automatically computing profits.

The system is composed of a Spring Boot backend API, a React web application, and an Android mobile application, all integrated through RESTful APIs to provide consistent and secure data access across platforms.

1.2 Objectives

1. Develop a functional restaurant management system focused on financial tracking and daily operations
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android Kotlin (mobile)
3. Design secure and maintainable RESTful APIs for cross-platform communication
4. Provide a simple, intuitive interface suitable for non-technical users
5. Deploy the system to a cloud-based environment for production and real usage
6. Enforce security through JWT authentication, BCrypt password hashing, and Role-Based Access Control (RBAC) for Admin and Tenant roles
7. Integrate features including external public APIs, Google OAuth login, file uploads, and SMTP email notifications.

1.3 Scope

Included Features:

- User authentication via Google OAuth 2.0 and JWT-based session management
- User registration and login functionality

- User roles (Admin/Tenant) enforced at API and UI levels
- Daily sales recording with categorized entries and variable pricing
- Expense tracking with automated profit computation
- Personal financial management and business expense recording
- Full CRUD operations for major business entities
- End-of-day closing to finalize and lock daily records
- File upload functionality (images/PDFs) linked to expense or related records
- Integration with at least one external public API
- React web application for full system management
- Android mobile application for sales and expense entry
- Spring Boot REST API using layered architecture
- PostgreSQL relational database

Excluded Features:

- Customer ordering or Point-of-Sale (POS) functionality
- Advanced inventory automation
- Online payment processing
- Data export to PDF or CSV formats
- Advanced analytics dashboards
- Push notifications
- Offline functionality
- Multi-tenant staff registration or employee-level sub-accounts

1.0 INTRODUCTION

1.1 Purpose

This document defines the system design for RevnU, a restaurant management application intended for restaurant owners (Tenants) to digitize their financial tracking. It serves as a reference for system functionality, architecture, and the scope of administrative and tenant operations.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: RevnU

Domain: Restaurant Management / Single-Tenant Financial Systems

Primary Users:

1. System Administrators (Admin)
2. Restaurant Owners (Tenants)

Problem Statement: Many small restaurants and food establishments rely on manual methods such as notebooks or spreadsheets to record daily sales, expenses, and employee salaries. These processes are prone to human error, lack data consistency, and make it difficult to generate accurate financial summaries or monitor business performance effectively.

Solution: RevnU provides a centralized digital platform that automates financial calculations, enforces structured data entry, and generates real-time daily and monthly summaries. The system integrates authentication, role-based authorization, file handling, external API consumption, and automated email notifications to enhance operational efficiency and data reliability.

2.2 Core User Journeys

Journey 1: Secure Login & Role Identification

1. User registers or logs in using email/password or Google OAuth
2. System validates credentials and generates a JWT token
3. The backend identifies the user role as Admin or Tenant.
4. The dashboard interface loads with permissions restricted to Admin or Tenant functionalities.
5. Dashboard loads based on role permissions

Journey 2: Sales Recording

1. The Tenant logs into the system using Google OAuth or standard credentials and selects the Sales icon from the sidebar navigation.
2. The Tenant clicks the + Add Sales button to open the "Add Sale Entry" form overlay at the bottom of the screen.
3. The Tenant enters the transaction amount and selects a pre-defined category (e.g., Meal, Drinks) using "Hot Key" chips designed for quick entry during busy establishment hours.

4. Upon submission, the system validates the input (checking for required fields) and uses the authenticated JWT to automatically link the record to the Tenant's unique restaurant_id, ensuring the data remains isolated from other tenants.
5. The new entry appears at the top of the sales list, and the Total Sales dashboard widget recalculates immediately to show the most current revenue status.

Journey 3: Expense Recording with File Upload

1. The Tenant logs into the RevnU web or mobile portal and selects the Expenses module from the sidebar navigation to view the operational cost ledger. Step 2: Entry Initiation
2. The Tenant clicks the + Add Expense button, which triggers a form to input the cost details.
3. The Tenant enters the amount and selects a category (e.g., Ingredients, Utilities, or Maintenance). Optional remarks are added to provide context for the specific purchase.
4. The Tenant uses the mobile camera or web file picker to upload a digital copy of the receipt (Image or PDF).
5. The system processes the multipart file, generates a unique entry in the Files table, and links the file_id to the new Expense record. The backend uses the Owner's JWT to ensure the expense is strictly associated with their specific restaurant_id.
6. The entry is saved to the database, and the Net Profit widget on the main Dashboard automatically updates by subtracting the new expense from the day's total revenue.

Journey 4: Staff Management & Payroll

1. Tenant navigates to the Staff Management module on the web dashboard.
2. Tenant creates a staff profile (Name, Position, Salary Rate) as an internal record without granting them system access.
3. Tenant records a payout for a specific staff member using the **Payout** interface.
4. System logs the payment as a labor expense and updates the establishment's net profit in real-time.

Journey 5: Restaurant Profile & Configuration Setup

1. After registration, the Tenant navigates to the Restaurant Profile section in the web application settings.

2. The Tenant enters establishment details such as the Restaurant Name, Physical Address, and Operating Hours.
3. The Tenant uploads a Business Logo (image file); the system stores this file and links the URL to the restaurants table record.
4. The Tenant defines custom Sales and Expense categories to streamline daily data entry.
5. The system persists these settings to the database, ensuring they are isolated to the Tenant's specific restaurant_id.

Journey 6: End-of-Day (EOD) Closing

1. Tenant ner reviews the aggregated daily sales, operational expenses, and staff payouts on the dashboard.
2. Tenant initiates the reporting process via Soft Closing.
3. System updates the status of the day's records to "finalized" to prevent accidental modifications.
4. Tenant calculates the final net profit and triggers an SMTP email summary to the owner's Gmail.

Journey 7: External API Data Retrieval

1. The Owner opens the web or mobile dashboard to view their daily metrics
2. The system sends a request to a public Philippine National Holiday API to check if the current date is a recognized holiday.
3. If the date is a holiday, the dashboard displays a Holiday Status Badge next to the sales charts.
4. The Owner uses this external information to evaluate why sales might be higher or lower than usual (e.g., increased foot traffic during a national holiday).

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration and login (Email/Password + Google OAuth 2.0)
2. JWT authentication with BCrypt password hashing
3. /me endpoint for authenticated user identification
4. RBAC: ADMIN and TENANT roles with endpoint-level restrictions
5. Full CRUD: Sales, Expenses, and Staff
6. File upload and retrieval for restaurant logos and expense documentation
7. Profit computation (sales – expenses – salaries)

8. End-of-Day transaction closing with record locking
9. SMTP email notifications for account and EOD events
10. External API integration
11. React Web Application
12. Android Mobile Application (limited functionality)

SHOULD HAVE

1. Monthly financial summaries
2. Search and filtering for sales and expenses
3. Structured validation and user-friendly error handling

COULD HAVE

1. Basic inventory tracking
2. Export functionality (PDF/CSV)

WON'T HAVE

1. POS and online payment processing
2. Advanced analytics dashboards
3. Push notifications
4. Staff-level registration or individual employee login credentials

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me
- **Security:** BCrypt password hashing, Google OAuth token validation, JWT issuance and validation, Secure session management

Feature: Role-Based Access Control (RBAC)

- **Screens:** Dashboard (Conditional view based on Admin or Tenant).

- **Functions:** ADMIN: Platform-level oversight and user management; TENANT: Exclusive access to their restaurant's logs, configuration, and reporting
- **API Endpoints:** GET /api/admin/users, PUT /api/admin/users/{userId}/status, PUT /api/admin/users/{userId}/role
- **Security:** Spring Security @PreAuthorize with role permissions

Feature: Establishment & Setup Dashboard

- **Screens:** Guided Configuration
- **Display:** Define business name, physical address, operating hours, and custom Sales/Expense categories.
- **API Endpoints:** POST /api/settings/business, POST /api/categories, GET /api/categories
- **Validation:** Progress checklist to guide the Tenant through full profile completion

Feature: Sales Management

- **Screens:** Sales Entry, Sales Summary.
- **Display:** Add, view, and update daily sales records.
- **API Endpoints:** POST /api/sales, GET /api/sales?date={date}, PUT /api/sales/{id}, DELETE /api/sales/{id}
- **Validation:** Mandatory amount and category fields; daily record locking after EOD.

Feature: Expense Management & File Upload

- **Screens:** Expense Entry, Receipt Viewer.
- **Functions:** Record operational costs with optional camera-based receipt photo uploads
- **API Endpoints:** POST /api/expenses, GET /api/expenses, POST /api/expenses/{id}/upload
- **Security:** Multipart file handling; receipt images linked to database records and stored in cloud/server storage

Feature: Staff & Salary Management

- **Screens:** Staff Directory, Payout History.
- **Data:** Allows the Tenant to create and manage internal records for employees to track business labor costs without granting them system access.

- **Data Collected:** Staff Name, Position, and Salary Rate.
- **API:** POST /api/staff, GET /api/staff, PUT /api/staff/{id}, DELETE /api/staff/{id}, POST /api/salaries.
- **Logic:** Salary payouts are recorded by the Tenant and automatically subtracted from the establishment's gross revenue to compute accurate net profit.

Feature: End-of-Day (EOD) reporting

- **Screens:** Daily Summary Dashboard.
- **Functions:** Soft Closing and Status Updates to finalize daily reports and compute net profit
- **API Endpoints:** POST /day/close, GET /day/summary/{date} .
- **Integration:** Triggers an automated SMTP email containing the daily financial report to the Tenant

Feature: External API Integration

- **Screens:** Dashboard, Sales Summary.
- **Functions:** Consume public holiday data to provide context for sales performance
- **API Endpoints:** GET /api/external/holidays
- **Display:** Visual holiday badge displayed alongside internal financial metrics to explain sales trends

Feature: Automated Email Notifications (SMTP)

- **Trigger:** Account creation and End-of-Day closing.
- **Functions:** Send automated Welcome emails and digital daily summary receipts.
- **Security:** Integration with an SMTP server to ensure deliverability.

2.5 Acceptance Criteria

AC-1: Successful User Registration & OAuth Login

- Given I am a new user
- When I enter a valid email and a strong password, or use the "Sign-In with Google" option
- And the system successfully validates the credentials or Google token
- Then my account should be created and saved in the database with a hashed password (BCrypt)

AC-2: Daily Sales Entry & Data Linking

- Given I am logged in as a Tenant with a valid JWT session
- When I record sales for the day with a specific category and amount
- Then the system should accurately update daily totals
- And the record must be linked to my specific restaurant_id to ensure data is isolated from other tenants

AC-3: Role-Based Access Control (RBAC)

- Given I am logged in with a "Tenant" role
- When I attempt to access the Admin user management dashboard
- Then the system should deny access with a "403 Forbidden" error at both the UI and API levels
- Given I am logged in with an Admin role
- When I attempt to record daily sales or initiate a Tenant's EOD report
- Then the system should deny access with a "403 Forbidden" error

AC-4: Expense Tracking with File Upload

- Given I have a physical receipt for a business expense
- When I create an expense entry and upload an image or PDF of the receipt
- Then the file should be stored on the server and linked to that specific expense record in the PostgreSQL database
- And I should be able to view or download the file later for verification

AC-5: Internal Staff Management & Payroll

- Given I am logged in as a Tenant
- When I create a staff record (non-user) and log a salary payout
- Then the system should store the payout as a labor expense linked to the establishment
- And the payout amount must be subtracted from the gross revenue during profit computation

AC-5: End-of-Day Closing & Automated Email

- Given all sales and expenses for the day are recorded
- When the Admin initiates the "End-of-Day reporting"
- Then the system should perform a Soft Closing, marking records as finalized and calculating the final net profit

- And the system must automatically send a summary notification email (SMTP) to the owner's registered Gmail

AC-6: External API Integration

- Given I am viewing the Sales Management or Dashboard screen
- When the page loads
- Then the system should fetch real-time holiday data from a public API
- And display a holiday status badge that provides meaningful context for the day's sales performance

AC-7: Secure Data Handling (DTO Implementation)

- Given the system processes a request for user or financial data
- When the API returns a response
- Then the system must utilize Data Transfer Objects (DTOs) to ensure sensitive information, such as password hashes, is never exposed

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time ≤ 2 seconds for CRUD operations.
- End-of-Day (EOD) locking and summary generation ≤ 3 seconds
- Support at least 50 concurrent users
- Database queries ≤ 500 ms
- Caching for External API data to maintain UI fluidity.
- File uploads (receipts) processed within reasonable time limits

3.2 Security Requirements

- HTTPS encryption for all client-server communications.
- JWT-based authentication
- Password hashing using bcrypt
- Role-Based Access Control (RBAC) for Admin and Tenant roles
- Input validation and SQL injection prevention
- File upload type and size restrictions
- Mandatory use of DTOs to prevent sensitive data exposure

3.3 Compatibility Requirements

- Web: Chrome, Firefox, Edge (latest versions)
- Android: API Level 24+

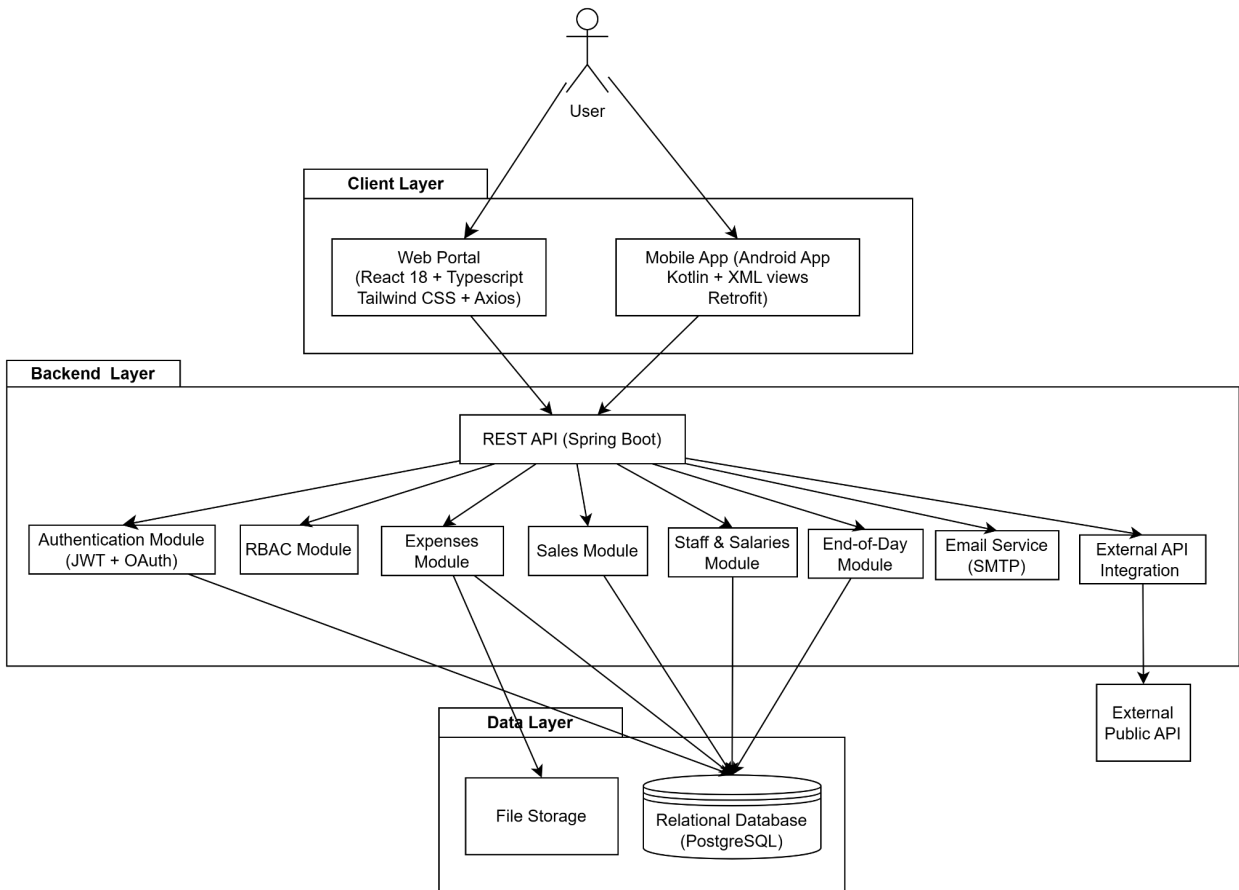
- Screen sizes: Mobile, tablet, desktop
- OS: Windows, macOS, Linux

3.4 Usability Requirements

- Minimal and intuitive interface
- Daily tasks completable within minutes
- Clear error and validation messages
- Consistent navigation across all screens

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security (JWT + OAuth 2.0), Spring Data JPA.
- **Database:** PostgreSQL 14+ (Relational Database).
- **File Storage:** Local Server Storage / Cloud Storage (for receipt images and logos).
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios.
- **Mobile Frontend:** Android Kotlin, XML views, Retrofit.
- **External Integrations:** Google OAuth 2.0, SMTP Server (Email), Public Philippine National Holiday API.
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android).
- **Deployment:** Render (Backend), Vercel (Web), APK Distribution (Mobile).

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": <email>, "password": <password>, "firstname": <firstname>, "lastname": <lastname>, }</pre>
Response Structure	<pre>{ "success": boolean, "data": { user { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, }, "accessToken": <token>, "refreshToken": <token>, } }</pre>

	<pre> }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>
--	---

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre> { "email": <email>, "password": <password>, } </pre>
Response Structure	<pre> { "success": boolean, "data": { "user": { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role>, }, "accessToken": <token>, "refreshToken": <token>, }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string } </pre>

Google OAuth Authentication

Description	Authenticates or registers users using Google OAuth 2.0 tokens.
API URL	/auth/google
HTTP Method	POST
Format	JSON for all requests/responses

Authentication	None
Request Payload	{ "idToken": <google_id_token> }
Response Structure	<pre> { "success": boolean, "data": { "user": { "email": <email>, "role": <role> }, "accessToken": <token> }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

Sales Entry

Description	Records a new sales transaction from the mobile application.
API URL	/sales
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Request Payload	{ "amount": <double>, "category": <string>, "note": <string> }
Response Structure	<pre> { "success": boolean, "data": { "saleId": <long>, "status": "recorded" }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

End-of-Day reporting

Description	Finalizes the daily ledger, locks records, and triggers the SMTP report
API URL	/day/close
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) - Owner/Manager only
Request Payload	<code>{"date": <yyyy-mm-dd> }</code>
Response Structure	<pre>{ "success": boolean, "data": { "summary": { "totalSales": <double>, "totalExpenses": <double>, "netProfit": <double> }, "reportSent": boolean }, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

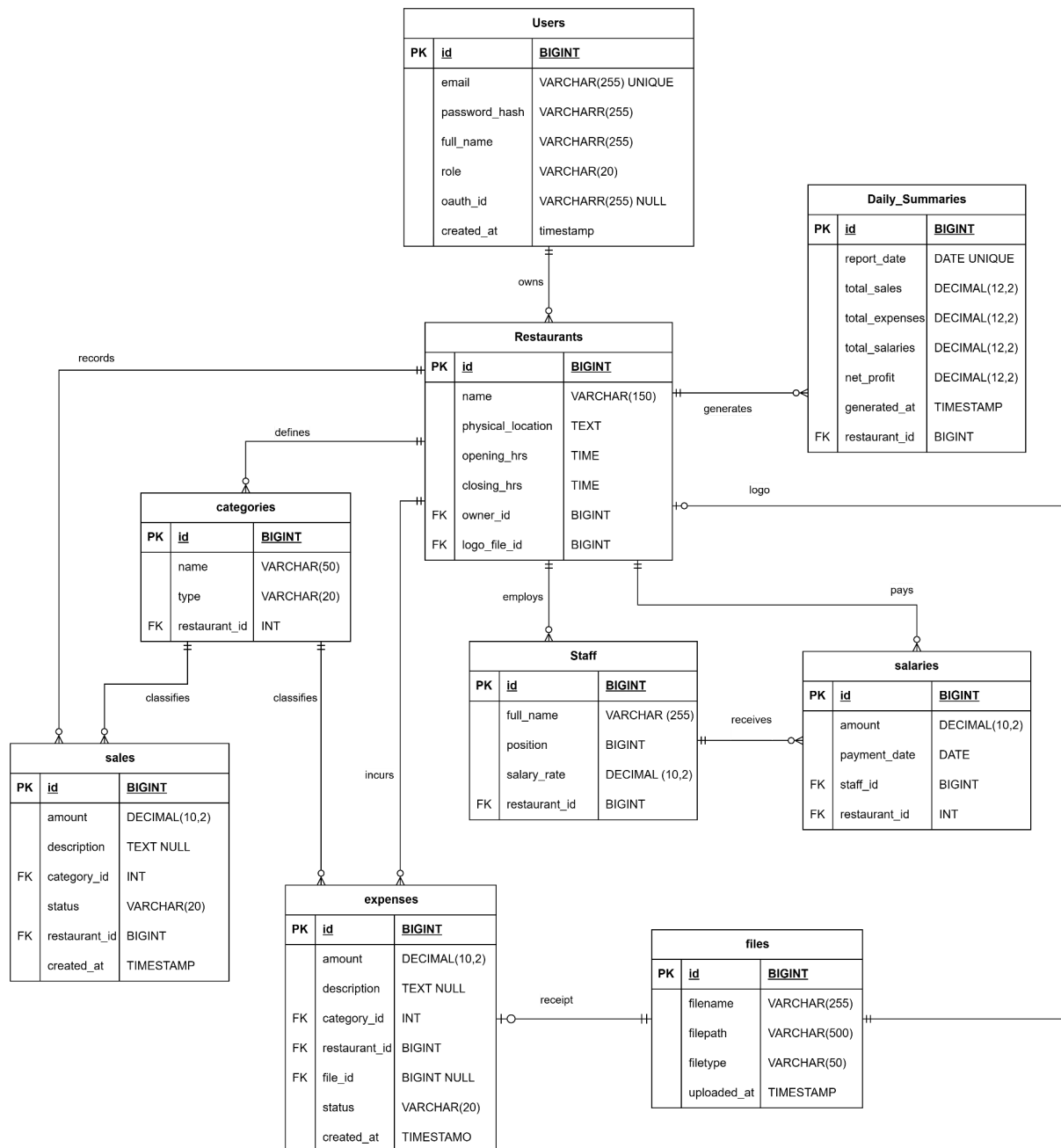
Example 1	<pre> json { "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } }, "timestamp": "2024-01-28T10:30:00Z" } </pre>

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many: Users → Restaurants:** A single user with the Tenant role owns and manages one restaurant establishment.
- **One-to-Many: Restaurants → Categories:** Each restaurant defines its own unique labels for sales and expenses to ensure tenant isolation.
- **One-to-Many: Restaurants → Staff:** The restaurant maintains internal records of employees (non-users) for labor tracking.
- **One-to-Many: Restaurants → Sales:** All revenue records are scoped to a specific establishment ID.
- **One-to-Many: Restaurants → Expenses:** All operational costs are linked to the establishment for profit computation.
- **One-to-Many: Staff → Salaries:** An internal staff record can have multiple salary payout logs over time.
- **One-to-One: Expenses ↔ Files:** Each expense record can optionally be linked to one digital receipt upload.
- **One-to-One: Restaurants ↔ Files:** A restaurant profile links to a specific file record for its business logo.
- **One-to-Many: Restaurants → Daily_Summaries:** An establishment generates one summary record per day during the EOD process.

Key Tables:

1. **users:** Stores account credentials, BCrypt hashed passwords, and Google OAuth links for Admins and Tenants.
2. **restaurants:** Global configuration including name, logo, location, and owner details.
3. **categories:** Custom labels for Sales and Expenses defined by the Tenant.
4. **staff:** Internal records of restaurant employees (non-users) used for payroll tracking.
5. **sales:** Revenue tracking records with status indicators for soft-closing.
6. **expenses:** Business cost records with links to digital receipt files.
7. **salaries:** Payout history for internal staff used in financial profit computations.
8. **files:** Storage metadata for uploaded business logos and receipt images.

9. **daily_summaries:** Historical record of daily financial performance and net profit.

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role (ADMIN/TENANT), oauth_id, created_at.
- **restaurants:** id, name, logo_file_id, physical_location, opening_hrs, closing_hrs, owner_id.
- **categories:** id, name, type (SALES/EXPENSE), restaurant_id.
- **staff:** id, full_name, position, salary_rate, restaurant_id.
- **sales:** id, amount, category_id, description, restaurant_id, status (OPEN/CLOSED), created_at.
- **expenses:** id, amount, category_id, description, restaurant_id, file_id, status (OPEN/CLOSED), created_at.
- **salaries:** id, amount, payment_date, staff_id, restaurant_id.
- **files:** id, filename, filepath, filetype, uploaded_at.
- **daily_summaries:** id, report_date, total_sales, total_expenses, total_salaries, net_profit, generated_at, restaurant_id.

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Login

Welcome Back to RevnU
Sign in to manage your restaurant's revenue and expenses.

Email
eg. staffname123 or 1234-56

Password
eg. Password@123
[Forgot Password?](#)

[Sign in](#)

[Sign in with Google](#)

By signing in, you agree to our [Terms of Service](#) and [Privacy Policy](#)

Don't have an account? [Register here](#)

Register

Create Your Account
Join the digital transformation for your restaurant.

[Register with Google](#)

— OR —

Full Name
eg. Juan Dela Cruz

Email
name@example.com

Password
(Minimum 8 characters)

Confirm Password
(Repeat your password)

[Create Account](#)

By registering, you agree to our [Terms and Privacy Policy](#)

Already have an account? [Sign in here](#)

Setup

Welcome, Owner!
Let's get your restaurant ready for its first shift.

Business Identity

Restaurant Name
eg. Juan Dela Cruz

Business Category
name@example.com

Physical Address
Minimum 8 characters

Opening Hours
Closing Hours
8:00 AM 10:00 PM

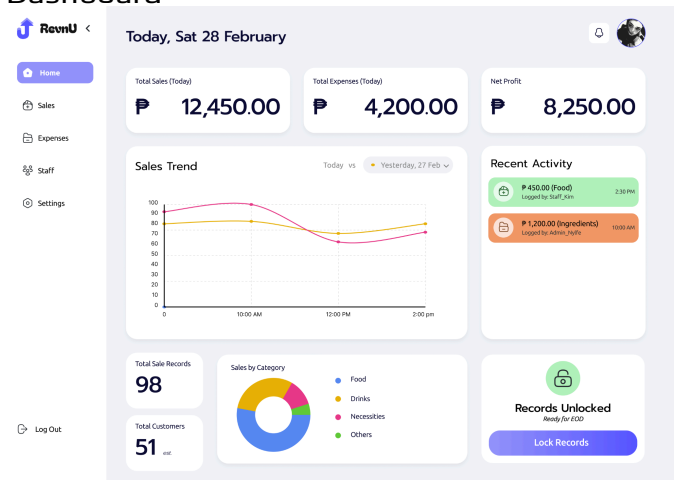
Business Logo
[Upload Business Logo](#)

Sales Categories
Define the items or services you sell (e.g., Meals, Drinks).
[Add](#)

Expense Categories
Define the items or services you sell (e.g., Meals, Drinks).
[Add](#)

☐ Require Receipt Photo

Dashboard



Sales Tracker

Sales [Lock Records](#) [Archived](#)

Search 150 records

1 of 5

[+ Add Sales](#)

Date	Amount	Category	Staff	Actions
03/01/2026 4:30 PM	385	Meal	Richelle Vallegos	Edit Delete
03/01/2026 4:23 PM	400	Meal	Richelle Vallegos	Edit Delete
03/01/2026 4:17 PM	100	Meal	Richelle Vallegos	Edit Delete
03/01/2026 4:00 PM	55	Drinks	Richelle Vallegos	Edit Delete
03/01/2026 3:40 PM	135	Drinks	Richelle Vallegos	Edit Delete
03/01/2026 3:15 PM	230	Meal	Richelle Vallegos	Edit Delete
03/01/2026 3:10 PM	75	Meal	Richelle Vallegos	Edit Delete
03/01/2026 3:04 PM	385	Meal	Richelle Vallegos	Edit Delete
03/01/2026 2:45 PM	105	Meal	Richelle Vallegos	Edit Delete
03/01/2026 2:40 PM	120	Meal	Richelle Vallegos	Edit Delete
03/01/2026 2:15 PM	150	Meal	Richelle Vallegos	Edit Delete

Add Sale Entry

Amount (in Pesos) Category Staff

Total Sales
Feb 28, 2026 2:58 PM
₱ 12,450.00

Expenses Tracker

Expenses [Archived](#)

Search 150 records

1 of 5

[+ Add Expense](#)

Date	Amount	Category	Receipt	Remarks	Actions
03/01/2026 5:00 PM	1000	Ingredient	fish_receipt.png	Blue Marlin Fish	Edit Delete
03/01/2026 12:35 PM	2000	Ingredient	rice_receipt.png	1 sack of Rice	Edit Delete
03/01/2026 12:35 PM	2200	Electricity	cebeco.png		Edit Delete

Add Expense Entry

Amount (in Pesos) Category Receipt (Optional)

Remarks (Optional)

Total Expenses
Feb 28, 2026 2:59 PM
₱ 4,200.00

Staff Management

RevnU

Home

Sales

Expenses

Staff

Settings

Log Out

Staff Management

Total Members: 10

Go To Payroll Table

Search

1 of 1

Name	Role	Email	Last Active	Actions
Richelle B. Vallejos	Cashier	chinchin@gmail.com	5 mins ago	Manage Payroll
Zach Vallejos	Waiter	nylfe@gmail.com	10 mins ago	Manage Payroll
Christian A. Mier	Chef	christian@gmail.com	5 hrs ago	Manage Payroll
Juan Dela Cruz	Waiter	juan@gmail.com	3 mins ago	Manage Payroll
Kim Serencio	Janitor	kim@gmail.com	45 mins ago	Manage Payroll

Staff Profile

RevnU

Home

Sales

Expenses

Staff

Settings

Log Out

Go back to Staff Management



Richelle B. Vallejos Cashier

chinchin@gmail.com
Joined: Apr 12, 2016

Profile Details

Home Address
Poblacon, Santander, Cebu

Birth Day
October 29, 1993

Email
chinchin@gmail.com

Emergency Person
Ethan Vallejos

Role
Cashier

Contact Number
09123456789

Join Date
April 12, 2016

Emergency Contact
09123456789

Salary History

Date	Period Covered	Amount	Method	Logged by
03/01/2026	02/15-02/28	6000	Cash	Admin

Remove From Restaurant

Payroll

RevnU

Home

Sales

Expenses

Staff

Settings

Log Out

Go back to Staff

Total Payroll (Month-to-Date)
₱ 12,450.00

Pending Payouts
3 staff

Next Scheduled Pay
Mar 15, 2026

Payroll

1 of 5

[+ Pay](#) [+ Bulk Pay](#)

Date	Employee Name	Amount	Period Covered	Status	Actions
03/01/2026	Richelle B. Vallejos	8000	02/15-02/28	PAID	Edit Delete
03/01/2026	Zach Vallejos	6000	02/15-02/28	PAID	Edit Delete
03/01/2026	Christian A. Mier	8000	02/15-02/28	PAID	Edit Delete
03/01/2026	Juan Dela Cruz	6000	02/15-02/28	PAID	Edit Delete
03/01/2026	Kim Serencio	5000	02/15-02/28	PAID	Edit Delete

Payroll-Bulk Pay

RevnU

Home

Sales

Expenses

Staff

Settings

Log Out

Go back to Expenses

Total Payroll (Month-to-Date)
₱ 12,450.00

Pending Payouts
3 staff

Next Scheduled Pay
Mar 15, 2026

Run Team Payroll

Period
Feb 15, 2026 Feb 28, 2026

Employee Name	Suggested Base Pay	Actual Payroll	Payment Method
Richelle B. Vallejos	8000	8000	CCash
Zach Vallejos	5000	6000	CCash
Christian A. Mier	8000	8000	Cash
Juan Dela Cruz	5000	6000	CCash
Kim Serencio	5000	6000	CCash

Late February Salaries

Cancel

Bulk Pay

Personal Settings

RevnU

Home

Sales

Expenses

Staff

Settings

Log Out

Settings

Personal Profile

Restaurant Profile

Personal Profile

Account Information

Full Name
Richemae V. Bigno

Email
richemae@gmail.com

Birth Day
8-15-2005

Contact Number
09497412507

Emergency Contact Person
Christian A. Mier

Change Password

Profile Picture


Change Profile Picture

Emergency Contact Person's Number
09680218179

Preferences & Notifications

☒ Email Notifications

☒ Push Notifications

Security & Connected Accounts

Connected Google Account
richemae@gmail.com

Restaurant Settings

RevnU

Home

Sales

Expenses

Staff

Settings

Log Out

Settings

Personal Profile

Restaurant Profile

Restaurant Profile

Business Identity

Restaurant Name
ChihyMc Restobar

Business Category
Restaurant

Physical Address
Liloan, Santander, Cebu

Opening Hours
8:00 AM

Closing Hours
10:00 PM

Business Logo


Change Business Logo

Sales Categories

Define the items or services you sell (e.g., Meals, Drinks).

Add

Expense Categories

Define the items or services you sell (e.g., Meals, Drinks).

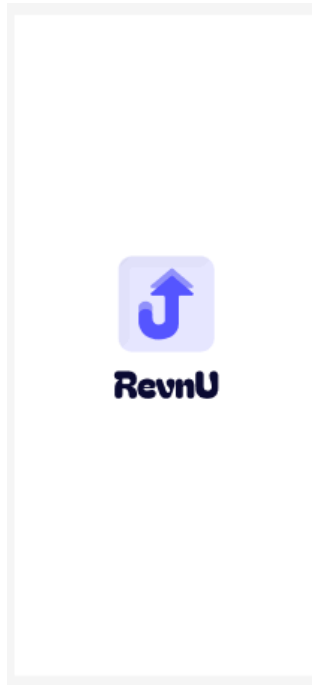
Add

☒ Require Receipt Photo

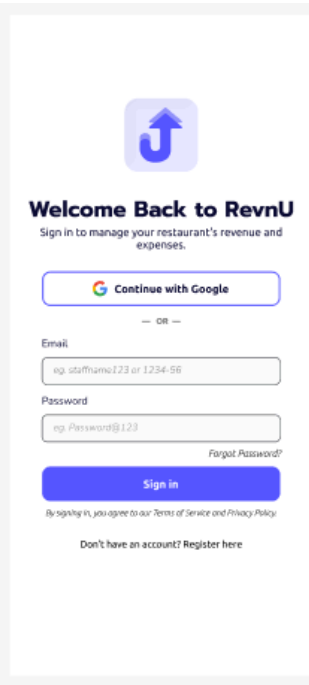
Danger Zone

7.2 Mobile Application Wireframes

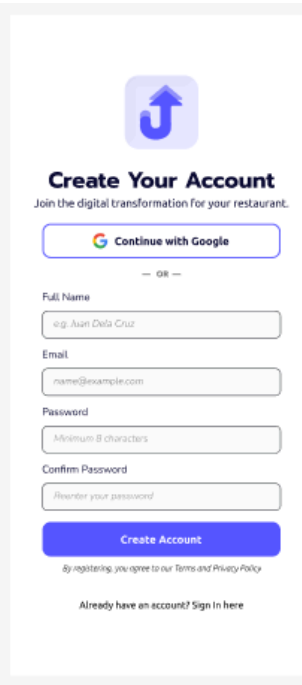
Splash



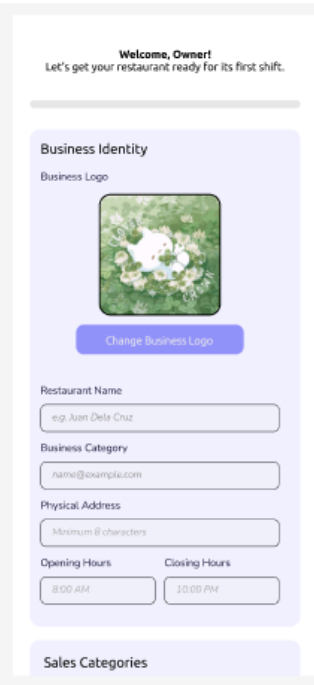
Login



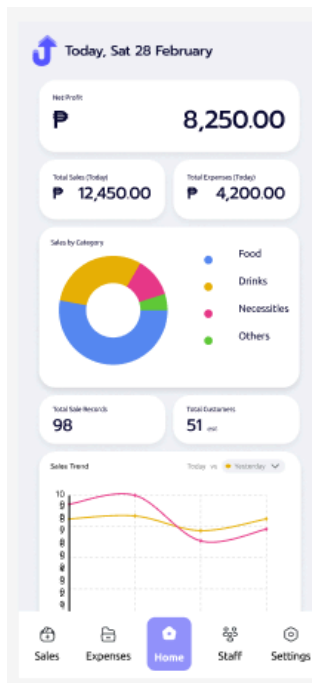
Registration



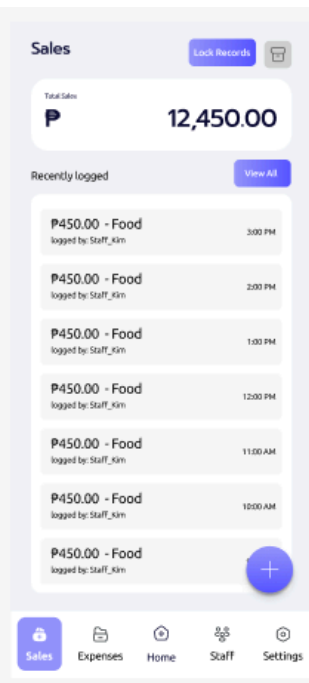
Onboarding-Tenant



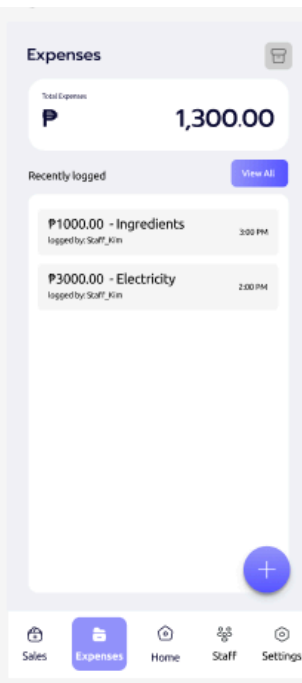
Dashboard



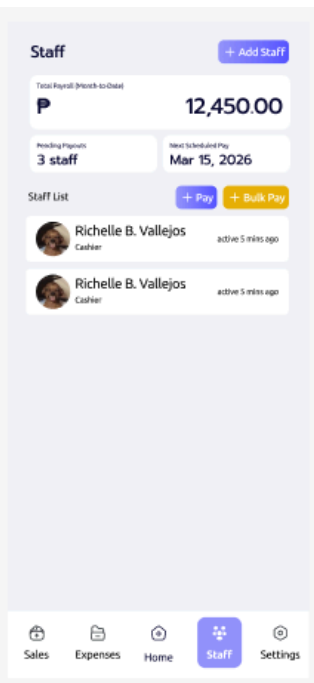
Sales



Expenses



Staff List



Staff Profile-History

← Staff Profile



Richelle B. Vallejos Cashier
chinchin@gmail.com
Joined: Apr 12, 2016

Profile Details

Salary History

₱ 6,000.00 (Feb 15-Feb 28)

03/01/2026

Issued by Admin (CASH)

Sales

Expenses

Home

Staff

Settings

Staff Profile-Details

← Staff Profile



Richelle B. Vallejos Cashier
chinchin@gmail.com
Joined: Apr 12, 2016

Profile Details

Salary History

Home Address

Poblacion, Santander, Cebu

Birthday

October 29, 1993

Contact Number

09123456789

Email

chinchin@gmail.com

Join Date

April 12, 2016

Emergency Person

Einan Vallejos

Emergency Contact

09123456789

Remove From Restaurant

Sales

Expenses

Home

Staff

Settings

Settings

Settings

Personal Profile

Restaurant Profile

Log Out

Sales

Expenses

Home

Staff

Settings

Personal Profile

← Personal Profile

Account Information

Profile Picture



Change Profile Picture

Full Name

e.g. Juan Dela Cruz

Email

name@example.com

Birthday

Minimum 8 characters

Contact Number

09123456789

Emergency Contact Person

Richard B. Blago

Sales

Expenses

Home

Staff

Settings

Restaurant Profile

← Restaurant Profile

Business Identity

Business Logo



Change Business Logo

Restaurant Name

e.g. Juan Dela Cruz

Business Category

name@example.com

Physical Address

Minimum 8 characters

Opening Hours

Closing Hours

8:00 AM

10:00 PM

Sales Categories

Sales

Expenses

Home

Staff

Settings

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup; Define Admin and Tenant boundaries and multi-tenant data flow.

Day 3-4: Complete FRS focusing on Sales, Expense, and Staff logging; define NFRs.

Day 5-7: Finalize three-tier system architecture: Spring Boot + React + Android.

Week 2: Detailed Design

Day 1-2: API Specification for Sales, Expenses, Staff, and EOD Reporting endpoints.

Day 3-4: Database Design for 5+ mandatory tables including Establishments and Internal Staff.

Day 5-6: UI/UX Wireframes for the Mobile Sales Keypad and Web Setup Dashboard.

Day 7: Implementation plan and ERD finalization.

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation & Identity

Day 1: Spring Boot setup with Security and JPA dependencies.

Day 2: Database configuration and Entity mapping for Tenant, Sale, Expense, and Staff.

Day 3: Google OAuth 2.0 Integration and RBAC for Admin and Tenant roles.

Day 4: Tenant onboarding logic and Establishment Profile creation.

Day 5: Tenant-level configuration and `/me` endpoint implementation.

Week 4: Core Business Logic

Day 1: Sales Logging API with Category filtering and tenant isolation.

Day 2: Expense Tracking API with Multipart image upload support for receipts.

Day 3: Category Management for custom Sales and Expense types.

Day 4: Internal Staff CRUD and Salary Payout logic for profit computation.

Day 5: SMTP Email service setup and API testing using Postman.

Phase 3: Web Application (Week 5-6)

Week 5: Management Hub

Day 1-2: React setup and Setup Dashboard for Owner identity and categories.

Day 3: User Management Screen for Admin-level tenant oversight.

Day 4: Sales & Expense Master Ledgers with filtering and Staff Directory.

Day 5: Live Analytics Dashboard with charts for Revenue, Expenses, and Net Profit.

Week 6: Advanced Features

Day 1: EOD Soft Closing workflow and SMTP Daily Report generation.

Day 2: Salary payout logging interface for internal staff tracking.

Day 3: External API Integration for displaying context-aware holiday data.

Day 4: Responsive design polish for tablet and desktop views.

Day 5: End-to-end integration testing and bug fixes.

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Sales Client

Day 1: Android Studio setup and Retrofit service layer for API calls.

Day 2: Google Sign-In integration for the mobile client.

Day 3: The Sales Keypad UI with large numeric buttons and Category chips.

Day 4: Expense Logger with Camera intent for digital receipt capture.

Day 5: Daily logs view for real-time revenue tracking.

Week 8: Experience & Polish

Day 1: Internal Salary History view for establishment labor tracking.

Day 2: UI haptics and "Success" feedback for rapid sales entry.

Day 3-4: Real-world testing on physical Android devices.

Day 5: APK generation and mobile technical documentation.

Phase 5: Integration & Deployment (Week 9-10)

Week 9: System Hardening

Day 1: Integration testing: ensuring Mobile entries sync with the Web Dashboard.

Day 2-3: Security review ensuring Tenants cannot access Admin endpoints.

Day 4: Performance testing for API response times under load.

Day 5: Documentation updates and Final System Audit.

Week 10: Deployment

Day 1: Backend deployment (Railway/Render) with PostgreSQL.

Day 2: Web app deployment (Vercel) and Domain setup.

Day 3: Final APK distribution.

Day 4: Live smoke Testing in "Solo" Owner mode.

Day 5: Project Submission.

Milestones:

- **M1 (End Week 2):** Design docs & Figma prototypes for Admin/Tenant flows complete.
- **M2 (End Week 4):** Backend API supports Auth and CRUD for Sales/Expenses.
- **M3 (End Week 6):** Web portal handles Tenant Setup, and Soft Closing reporting.
- **M4 (End Week 8):** Mobile app functional for fast sales entry & receipt capture.
- **M5 (End Week 10):** Full system live; SMTP reports sending daily to Tenant.

Critical Path:

1. Google OAuth & First-User Logic (Week 3)
2. Sales/Expense Transaction API (Week 3-4)
3. Mobile Sales Keypad UI (Week 7)
4. Web Setup Dashboard (Week 5)
5. SMTP Daily Report Engine for Tenants (Week 6)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements